

```
import gensim.downloader as api
import numpy as np
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_899/3408331118.py in <cell line: 0>()
----> 1 import gensim.downloader as api
      2 import numpy as np
```

ModuleNotFoundError: No module named 'gensim'

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either `!pip` or `!apt`.

To view examples of installing some common dependencies, click the "Open Examples" button below.

OPEN EXAMPLES

```
!pip install gensim
```

Collecting gensim

Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.2)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
27.9/27.9 MB 61.8 MB/s eta 0:00:00

Installing collected packages: gensim
Successfully installed gensim-4.4.0

```
import gensim.downloader as api
import numpy as np
```

```
print("Loading pretrained embeddings...")
model = api.load("glove-wiki-gigaword-100")
print("Model loaded successfully")

# -----
# Helper functions for WEAT
# -----

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def association(word, A, B):
    return np.mean([cosine_similarity(model[word], model[a]) for a in A]) - \
           np.mean([cosine_similarity(model[word], model[b]) for b in B])

def weat_effect_size(X, Y, A, B):
    assoc_X = np.array([association(x, A, B) for x in X])
    assoc_Y = np.array([association(y, A, B) for y in Y])

    mean_diff = np.mean(assoc_X) - np.mean(assoc_Y)
    std_dev = np.std(np.concatenate([assoc_X, assoc_Y]))

    return mean_diff / std_dev

# -----
# Test 1: Gender Bias
# -----

male_terms = ["man", "male", "boy", "brother", "he", "him"]
female_terms = ["woman", "female", "girl", "sister", "she", "her"]

career_words = ["career", "business", "office", "salary", "professional"]
family_words = ["home", "family", "children", "parent", "marriage"]

gender_bias_score = weat_effect_size(
    male_terms, female_terms, career_words, family_words
)

print("\nGender Bias WEAT Effect Size:", round(gender_bias_score, 3))

# -----
# Test 2: Racial Bias
# -----
```

```
europaan_names = ["adam", "harry", "josh", "alan", "frank"]
african_names = ["jamal", "leroy", "tariq", "darnell", "tyrone"]

pleasant_words = ["love", "peace", "friend", "joy", "wonderful"]
unpleasant_words = ["hate", "pain", "anger", "evil", "terrible"]

racial_bias_score = weat_effect_size(
    european_names, african_names, pleasant_words, unpleasant_words
)
print("Racial Bias WEAT Effect Size:", round(racial_bias_score, 3))
```

```
Loading pretrained embeddings...
[=====] 100.0% 128.1/128.1MB downloaded
Model loaded successfully

Gender Bias WEAT Effect Size: 0.742
Racial Bias WEAT Effect Size: 1.662
```